

Comparación entre Modelos de Proceso de Requisitos en Escenarios Reales

Díaz Steven
Tigasi Paul
Vera Anthony

12 de mayo de 2026

1. Introducción

Actualmente, el desarrollo de software requiere metodologías que permitan organizar adecuadamente las actividades relacionadas con la obtención y gestión de requisitos. La ingeniería de requisitos se considera una de las etapas más importantes dentro de un proyecto de software, debido a que los errores cometidos en esta fase pueden afectar directamente la calidad del sistema final [1].

A lo largo del tiempo han surgido distintos modelos de proceso de software, entre ellos el modelo en cascada, el modelo ágil y el modelo espiral. Cada uno posee características diferentes y se adapta a distintos escenarios dependiendo del tamaño del proyecto, la estabilidad de los requisitos y el nivel de riesgo existente [2].

En muchos casos reales, las empresas combinan metodologías tradicionales y ágiles para adaptarse a cambios constantes sin perder el control de la documentación y los requisitos. Un ejemplo de esto puede observarse en empresas como Ericsson, donde los procesos ágiles conviven con estructuras más tradicionales para gestionar proyectos complejos [3].

2. Modelo en Cascada

El modelo en cascada es considerado uno de los enfoques más tradicionales dentro de la ingeniería de software. Este modelo organiza el desarrollo en fases secuenciales, donde cada etapa debe completarse antes de iniciar la siguiente [2].

Entre las principales ventajas de este modelo se encuentra la facilidad para planificar actividades y mantener documentación organizada. Debido a esto, suele utilizarse en proyectos donde los requisitos son claros y poco propensos a cambios, como sistemas bancarios, administrativos o gubernamentales [7].

Sin embargo, diversos estudios señalan que el modelo en cascada presenta dificultades cuando existen cambios frecuentes en los requisitos, ya que regresar a fases anteriores puede generar retrasos y mayores costos [4]. Por esta razón, actualmente muchas organizaciones han reducido su uso en proyectos dinámicos.

3. Modelo Ágil

Las metodologías ágiles surgieron como una alternativa frente a las limitaciones de los modelos tradicionales. Su principal característica es la capacidad de adaptarse continuamente a cambios durante el desarrollo del software [5].

Según Abbas [4], el enfoque ágil permite trabajar mediante iteraciones pequeñas, facilitando la comunicación constante entre el cliente y el equipo de desarrollo. Esto ayuda a detectar errores rápidamente y mejorar las funcionalidades del sistema de manera progresiva.

En escenarios reales, las metodologías ágiles son ampliamente utilizadas en aplicaciones móviles, plataformas web y sistemas empresariales modernos debido a la rapidez con la que evolucionan las necesidades de los usuarios [3]. No obstante, algunos autores indican que este modelo puede generar dificultades cuando no existe suficiente documentación o participación activa del cliente [5].

4. Modelo Espiral

El modelo espiral fue propuesto por Barry Boehm como una metodología enfocada principalmente en el análisis y control de riesgos [6]. Este modelo combina características del desarrollo iterativo con procesos de evaluación continua.

Cada ciclo del modelo espiral incluye planificación, análisis de riesgos, desarrollo y validación. Gracias a esto, se considera adecuado para proyectos complejos y de gran tamaño, especialmente en sectores donde los errores pueden tener consecuencias importantes, como sistemas militares, aeroespaciales o de telecomunicaciones [2].

Aunque este modelo ofrece una buena gestión de riesgos, también requiere mayor experiencia técnica y administrativa para poder implementarse correctamente [8].

5. Cuadro Comparativo de Modelos

Aspecto	Modelo en Cascada	Modelo Ágil	Modelo Espiral
---------	-------------------	-------------	----------------

Forma de trabajo	Desarrollo secuencial y ordenado por fases [2].	Desarrollo iterativo e incremental basado en entregas pequeñas [5].	Desarrollo iterativo centrado en análisis de riesgos [6].
Flexibilidad ante cambios	Muy limitada debido a su estructura lineal [4].	Alta capacidad de adaptación a cambios frecuentes [5].	Moderada, dependiendo del análisis realizado en cada ciclo [8].
Participación del cliente	Participación reducida durante el desarrollo [7].	Participación constante mediante retroalimentación continua [4].	Participación periódica durante las evaluaciones del proyecto [6].
Documentación	Extensa y detallada [2].	Menor documentación y mayor enfoque en resultados funcionales [5].	Documentación moderada enfocada en riesgos y planificación [8].
Gestión de riesgos	Baja gestión de riesgos [7].	Los riesgos se reducen mediante iteraciones cortas [4].	Alta gestión y análisis continuo de riesgos [6].
Costo ante cambios	Los cambios suelen incrementar considerablemente el costo [2].	Los cambios pueden incorporarse de forma más sencilla [5].	Los cambios dependen del análisis de riesgos y planificación [8].
Escenarios recomendados	Proyectos con requisitos estables y bien definidos [7].	Aplicaciones web, móviles y proyectos dinámicos [3].	Proyectos complejos o críticos de gran tamaño [6].
Ventajas principales	Organización clara y control documental [2].	Rapidez, adaptabilidad y comunicación constante [4].	Mayor control de riesgos y evaluación continua [8].
Desventajas principales	Dificultad para manejar cambios durante el proyecto [4].	Dependencia de la colaboración continua del cliente [5].	Mayor complejidad administrativa y altos costos [6].

6. Aplicación de los Modelos en Escenarios Reales

En escenarios reales, la selección del modelo de proceso depende principalmente de las características del proyecto y del entorno donde se desarrollará el software. Estudios realizados en empresas tecnológicas muestran que muchas organizaciones combinan modelos tradicionales y ágiles para mejorar la gestión de requisitos [3].

Por ejemplo, en sistemas bancarios y gubernamentales todavía se utilizan modelos tradicionales debido a la necesidad de mantener documentación detallada y procesos más controlados [7]. Por otro lado, empresas que desarrollan aplicaciones móviles y plataformas web suelen utilizar metodologías ágiles debido a la rapidez con la que cambian los requisitos del mercado [5].

Además, investigaciones recientes muestran que algunos proyectos complejos utilizan enfoques híbridos donde metodologías ágiles trabajan dentro de entornos organizacionales tradicionales [9]. Esto demuestra que actualmente no existe un único modelo ideal para todos los proyectos.

7. Conclusiones

La comparación realizada permite observar que cada modelo de proceso de requisitos posee características específicas que lo hacen adecuado para determinados escenarios. El modelo en cascada continúa siendo útil en proyectos donde los requisitos son estables y existe poca probabilidad de cambios.

Por otra parte, las metodologías ágiles ofrecen mayor flexibilidad y rapidez frente a necesidades cambiantes, razón por la cual son ampliamente utilizadas en la actualidad. Finalmente, el modelo espiral destaca principalmente por su capacidad para gestionar riesgos en proyectos complejos.

También puede concluirse que muchas organizaciones modernas combinan elementos de distintos modelos con el objetivo de aprovechar sus ventajas y reducir sus limitaciones.

Referencias

- [1] S. España, N. Condori-Fernández, A. González y Ó. Pastor, “An empirical comparative evaluation of requirements engineering methods”, *Journal of the Brazilian Computer Society*, vol. 16, pp. 3–19, 2010. Disponible en: <https://link.springer.com/article/10.1007/s13173-010-0003-5>
- [2] Iqbal H. Sarker, Md. Faisal Faruque, Ujjal Hossen y Atikur Rahman, “A Survey of Software Development Process Models in Software Engineering”, *International Journal of Software Engineering and Its Applications*, vol. 9, no. 11, pp. 55–70, 2015. Dis-

ponible en: https://www.researchgate.net/publication/298656663_A_Survey_of_Software_Development_Process_Models_in_Software_Engineering

- [3] Ville T. Heikkilä et al., “Managing the requirements flow from strategy to release in large-scale agile development: a case study at Ericsson”, *Empirical Software Engineering*, vol. 22, pp. 2892–2936, 2017. Disponible en: <https://link.springer.com/article/10.1007/s10664-016-9491-z>
- [4] Jalil Abbas, “Quintessence of Traditional and Agile Requirement Engineering”, *Journal of Software Engineering and Applications*, vol. 9, pp. 63–70, 2016. Disponible en: <https://www.scirp.org/journal/paperinformation?paperid=64676>
- [5] Tazeen Fatima y Waqas Mahmood, “Requirement Engineering in Agile”, *International Journal of Education and Management Engineering*, vol. 9, no. 4, pp. 20–33, 2019. Disponible en: <https://www.mecs-press.org/ijeme/ijeme-v9-n4/v9n4-3.html>
- [6] Barry W. Boehm, “A Spiral Model of Software Development and Enhancement”, *Computer*, vol. 21, no. 5, pp. 61–72, 1988.
- [7] A. Govardhan, “A Comparison Between Five Models Of Software Engineering”, *International Journal of Computer Science Issues*, vol. 7, no. 5, pp. 94–101, 2010. Disponible en: https://www.researchgate.net/publication/258959806_A_Comparison_Between_Five_Models_Of_Software_Engineering
- [8] Manohar K. Kodmelwar et al., “A Comparative Study of Software Development Waterfall, Spiral and Agile Methodology”, *Journal of Positive School Psychology*, vol. 6, no. 3, 2022. Disponible en: <https://journalppw.com/index.php/jpsp/article/view/4267>
- [9] Rashidah Kasauli et al., “Agile Islands in a Waterfall Environment: Requirements Engineering Challenges and Strategies in Automotive”, 2021. Disponible en: <https://arxiv.org/abs/2106.11187>